

Informe del Modelo Arquitectónico 4+1 — Sistema de Turnos

Tecnologías: Java 21 + Spring Boot 4 (backend) · TypeScript + React 19 (frontend) · PostgreSQL 15 · Docker

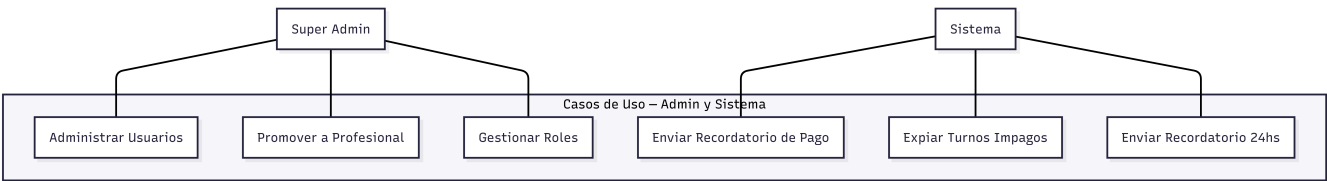
Introducción

El modelo **4+1** (Kruchten, 1995) organiza la arquitectura de software en cinco vistas concurrentes, cada una orientada a un stakeholder distinto. Este informe describe la arquitectura del Sistema de Turnos aplicando dicho modelo, con diagramas generados a partir del código y la configuración del proyecto.

1. Vista de Escenarios (Casos de Uso)

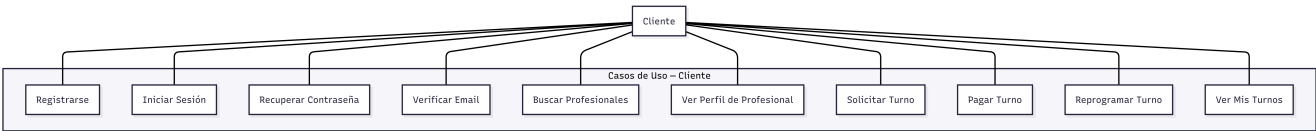
La vista de escenarios unifica las demás vistas al describir las funcionalidades principales del sistema. Se identifican tres actores principales.

Actor: Administrador / Sistema



El administrador y el sistema son responsables de tareas de gestión global: ABM de servicios, gestión de usuarios, cancelación de turnos, y procesos automáticos como el vencimiento de turnos no pagos y los recordatorios por email (scheduler).

Actor: Cliente (Paciente)



El cliente puede registrarse, iniciar sesión, solicitar un turno, reprogramarlo, cancelarlo, y consultar el historial de sus turnos. También puede realizar el pago del turno a través de un mock de pasarela de pagos.

Actor: Profesional

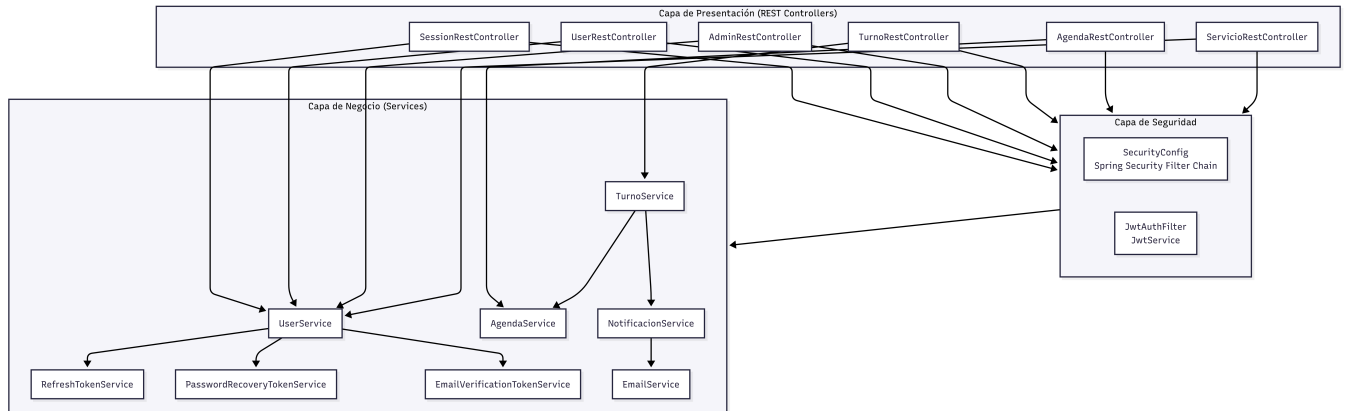


El profesional configura su agenda (días y horarios disponibles), define los servicios que ofrece, gestiona los turnos solicitados y puede tomar turnos para pacientes no registrados.

2. Vista Lógica

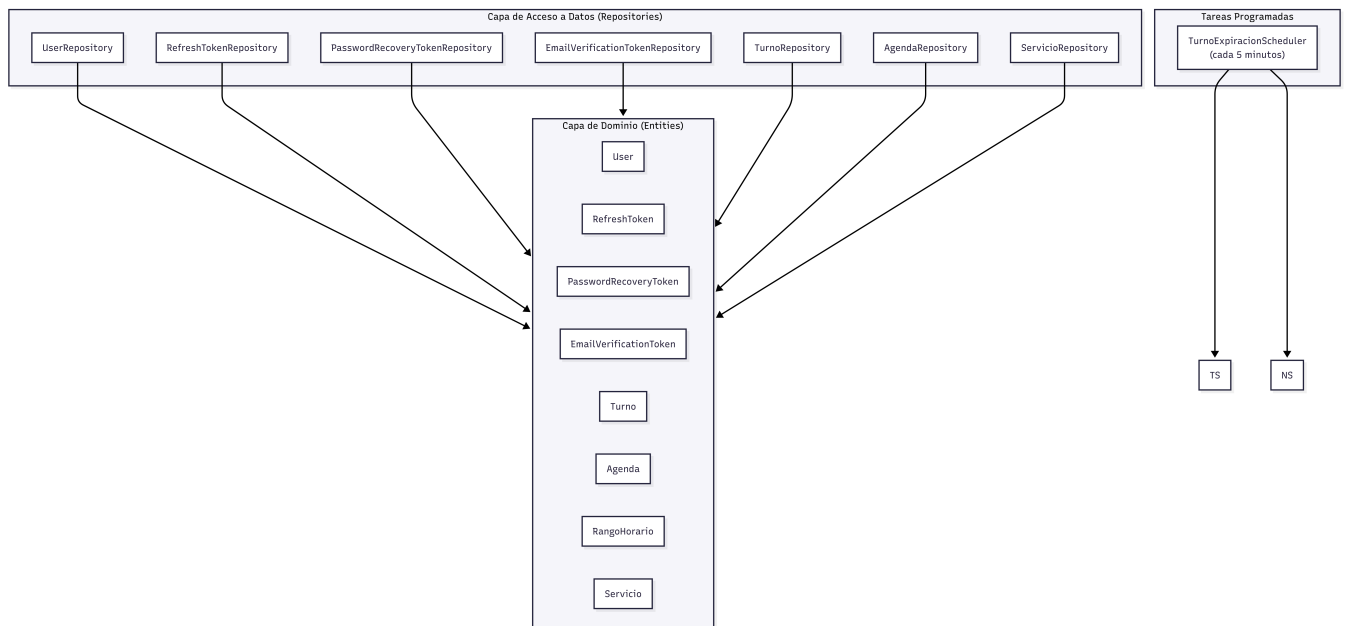
La vista lógica descompone el sistema en componentes y sus relaciones, reflejando la estructura orientada a objetos del backend y frontend.

Backend — Presentación, Seguridad y Servicios



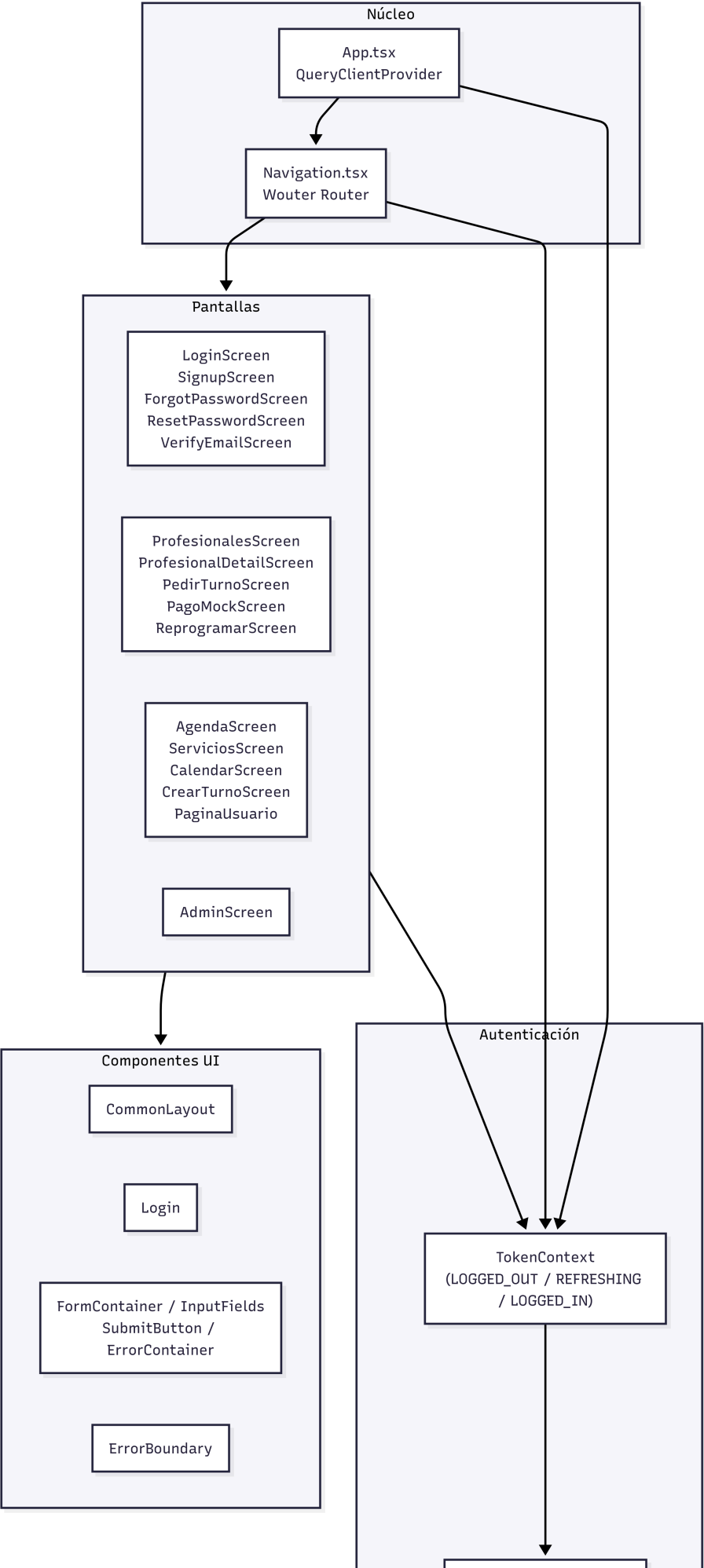
El backend sigue una arquitectura en capas con controladores REST (presentación), servicios de negocio y repositorios. Spring Security gestiona la autenticación con JWT y refresh tokens, mientras que los DTOs separan la representación externa del modelo interno.

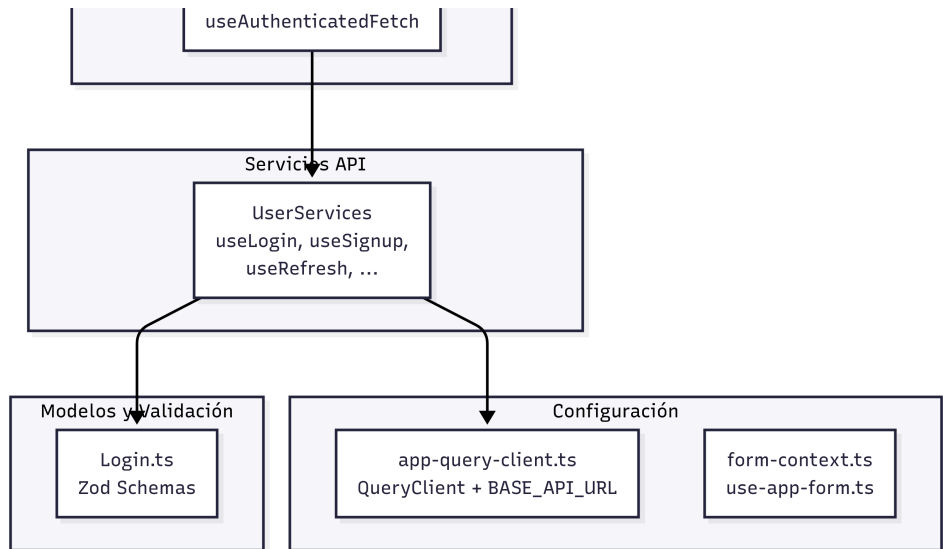
Backend — Datos y Tareas Programadas



Las entidades JPA reflejan el modelo de dominio (Usuario, Turno, Agenda, Servicio, Notificación). Un scheduler programado con `@Scheduled` ejecuta tareas periódicas: vencimiento de turnos impagos y envío de recordatorios 24 horas antes.

Frontend



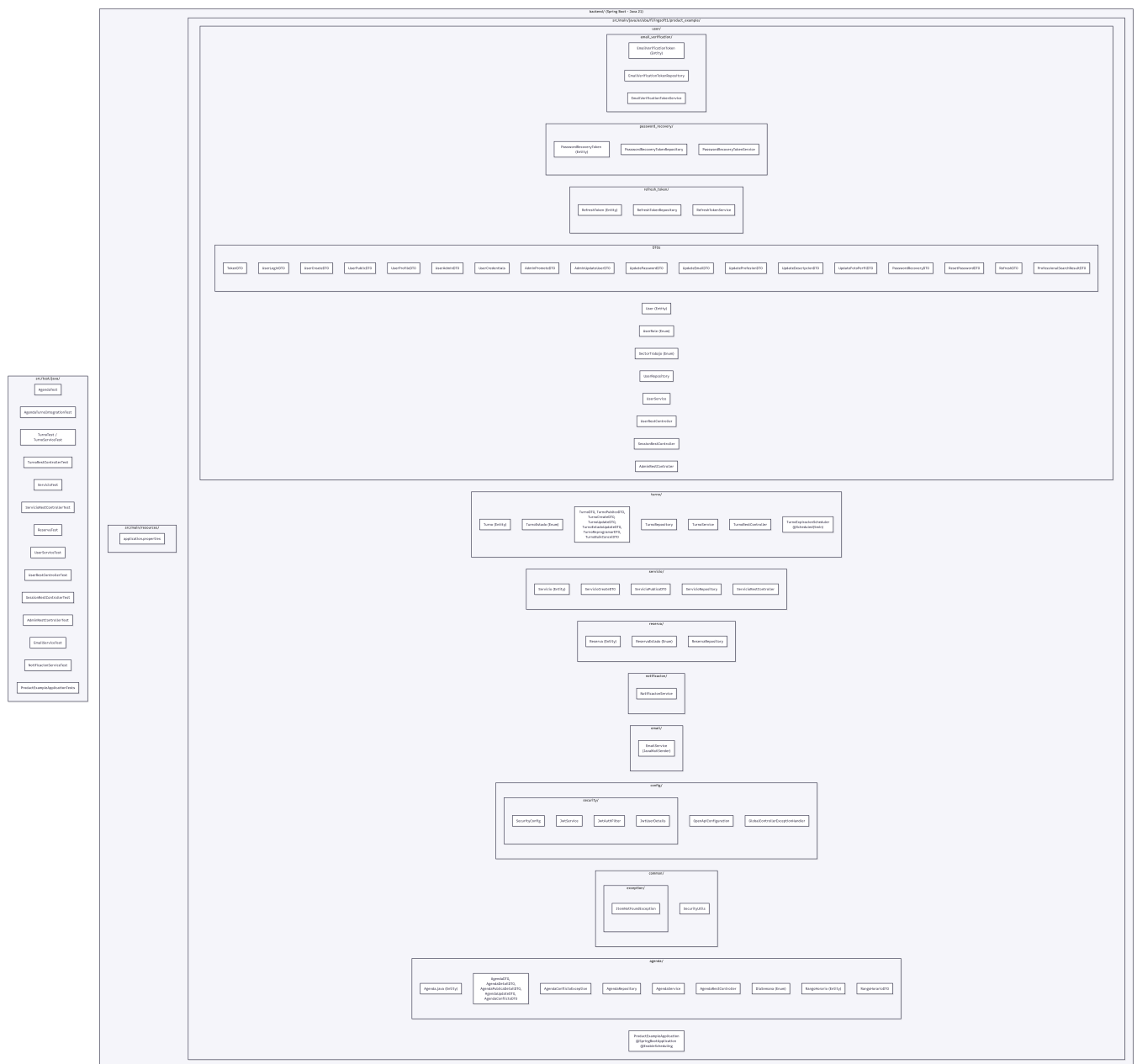


El frontend es una SPA en React con TanStack Query para fetching y caché del lado del cliente. Los componentes de UI se organizan en pantallas (screens), componentes reutilizables (components) y un contexto de autenticación (TokenContext). El enrutamiento está a cargo de Wouter.

3. Vista de Desarrollo (Estructura de Paquetes)

La vista de desarrollo muestra la organización del código fuente desde la perspectiva del programador, incluyendo módulos, dependencias y estructura de paquetes.

Backend



El backend se estructura por funcionalidad (package-by-feature): `agenda`, `turno`, `user`, `reserva`, `servicio`, `email`, `notificacion`. Cada paquete contiene su propio controlador, servicio, entidad, repositorio y DTOs. La configuración de seguridad reside en `config.security`.

Frontend

frontend/ (React 19 + TypeScript + Vite)

src/

screens/

LoginScreen

SignupScreen

ForgotPasswordScreen

ResetPasswordScreen

VerifyEmailScreen

PendingVerificationScreen

ProfesionalesScreen

ProfesionalDetailScreen

PedirTurnoScreen

PagoMockScreen

ReprogramarScreen

AgendaScreen

CalendarScreen

ServiciosScreen

CrearTurnoScreen

AdminScreen

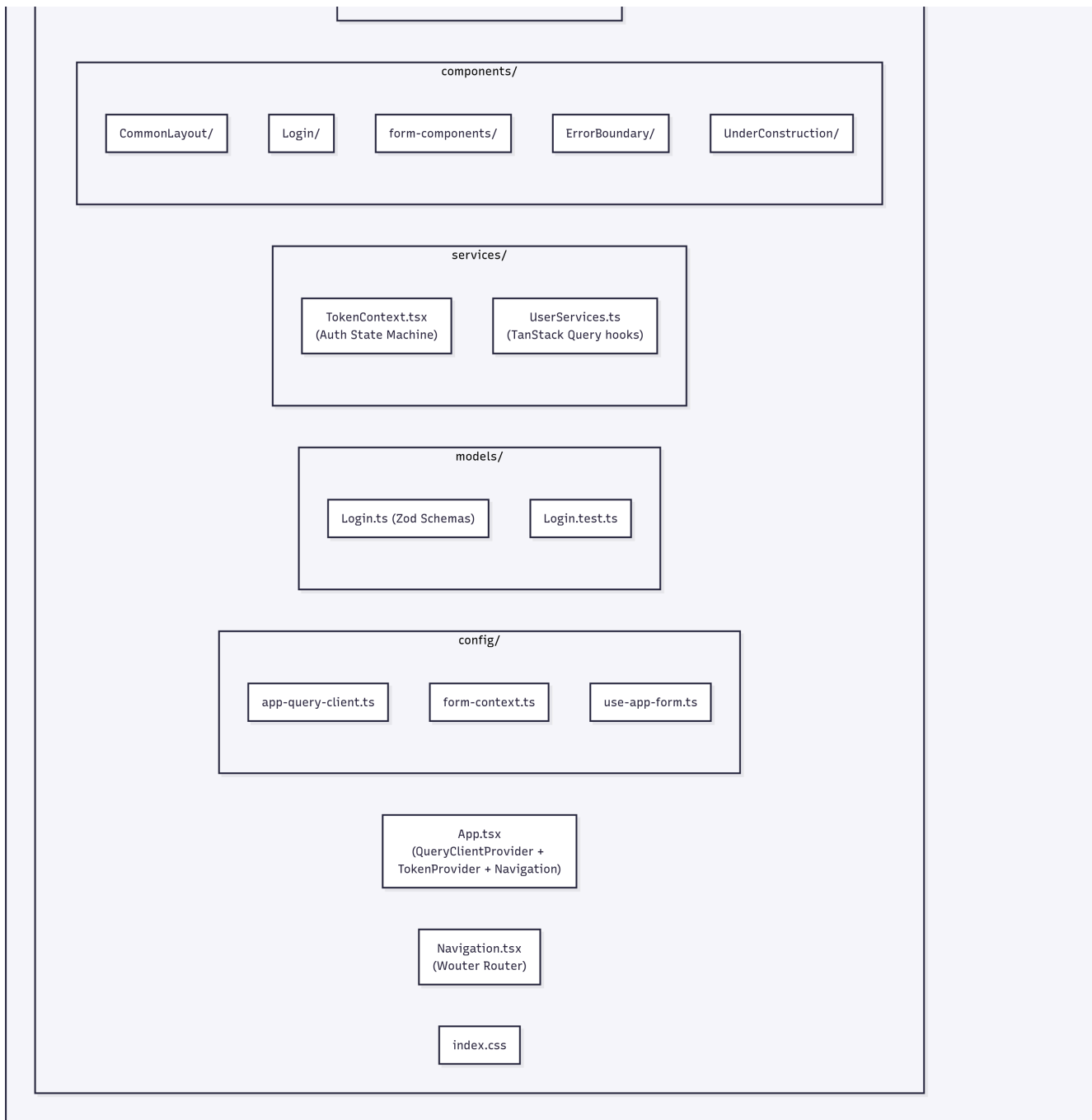
PaginaUsuario

CambioPasswordLogueado

CambioEmailLogueado

CambioProfesionLogueado

main.tsx
index.html
vite.config.ts

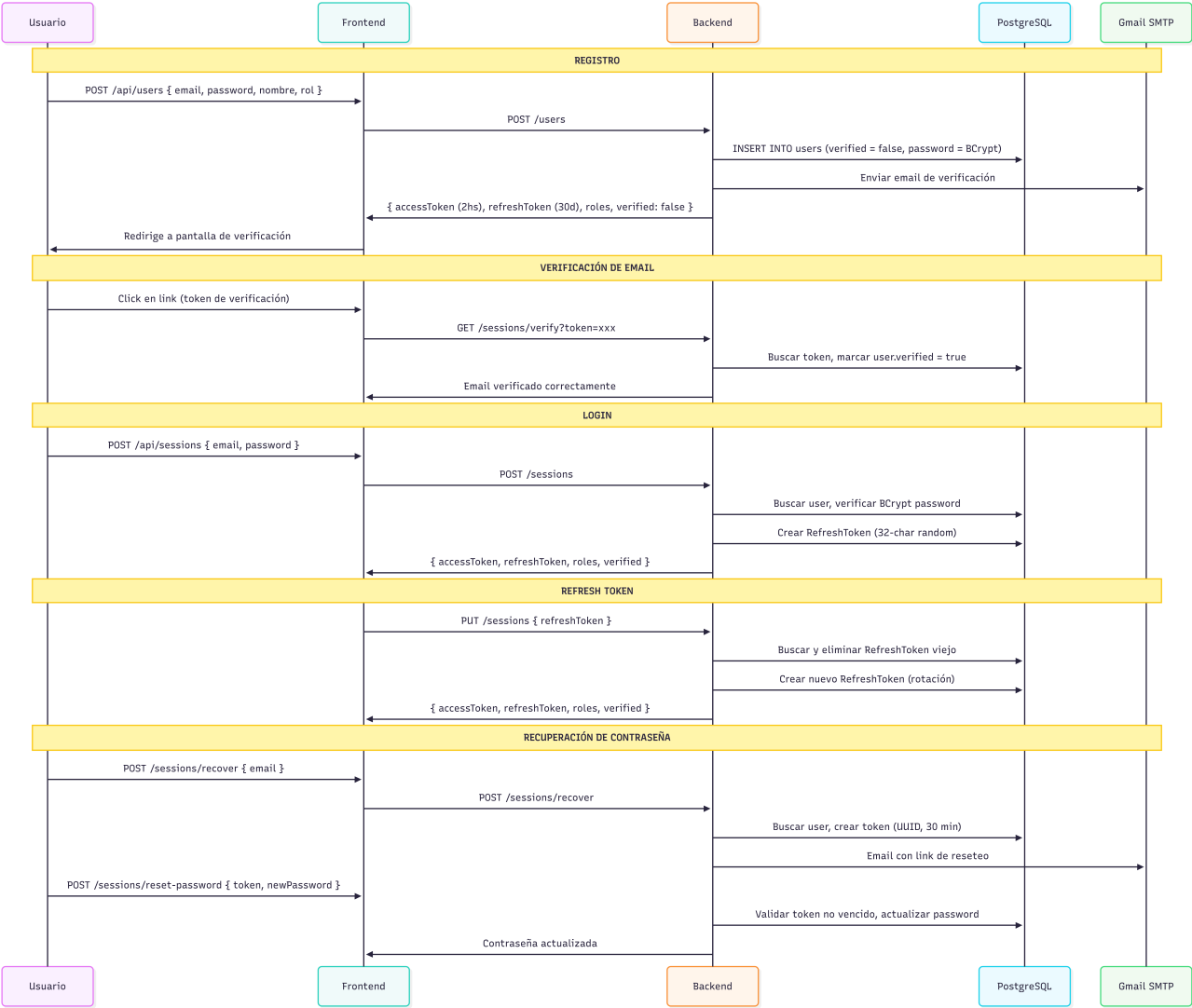


El frontend se organiza en `screens/` (páginas), `components/` (componentes reutilizables), `services/` (capa de API), `hooks/`, `models/`, `config/` y `utils/`. Esta separación favorece la mantenibilidad y el testing unitario.

4. Vista de Procesos

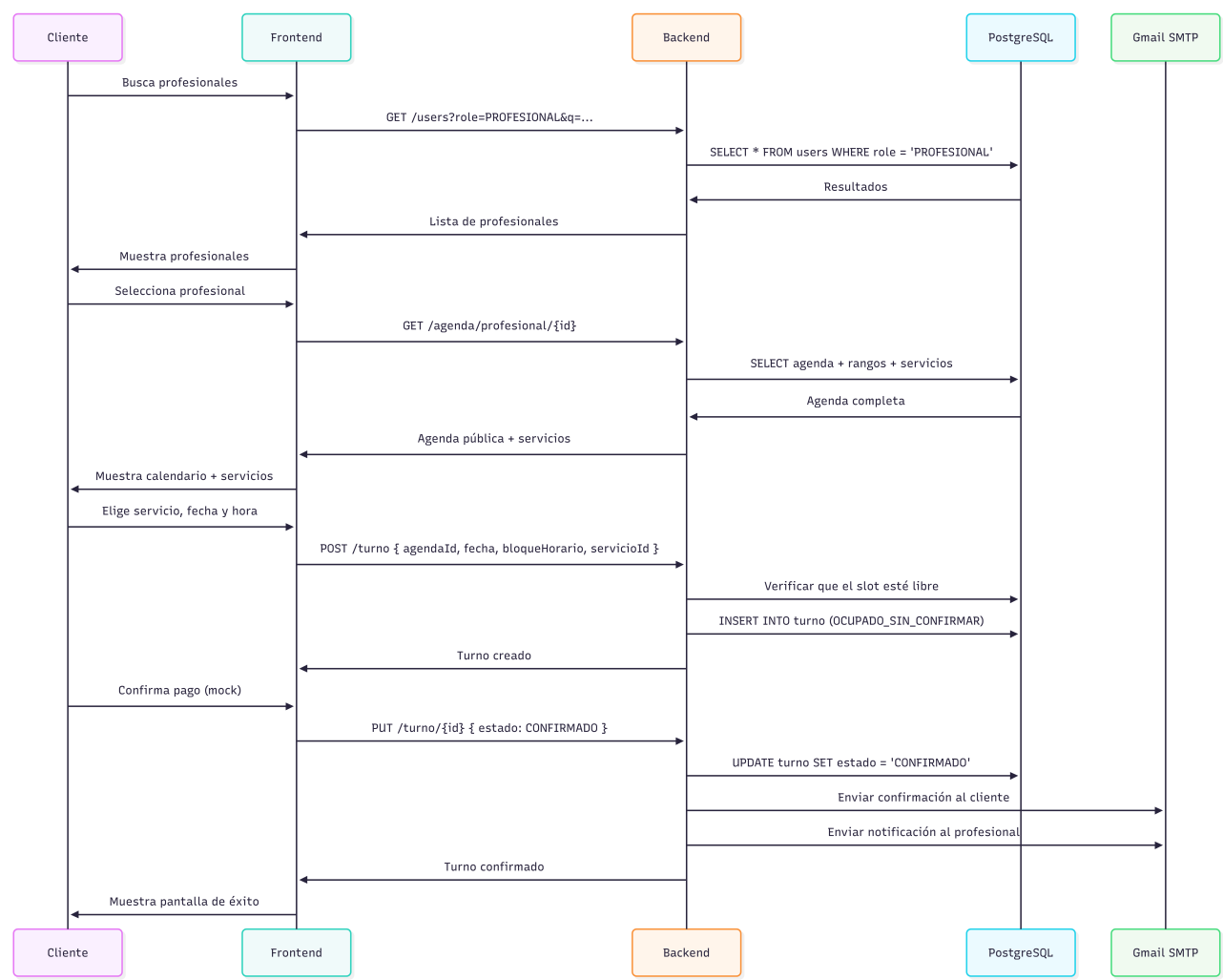
La vista de procesos describe los flujos de ejecución concurrente, la comunicación entre componentes y los aspectos de concurrencia del sistema.

Flujo de Autenticación y Seguridad



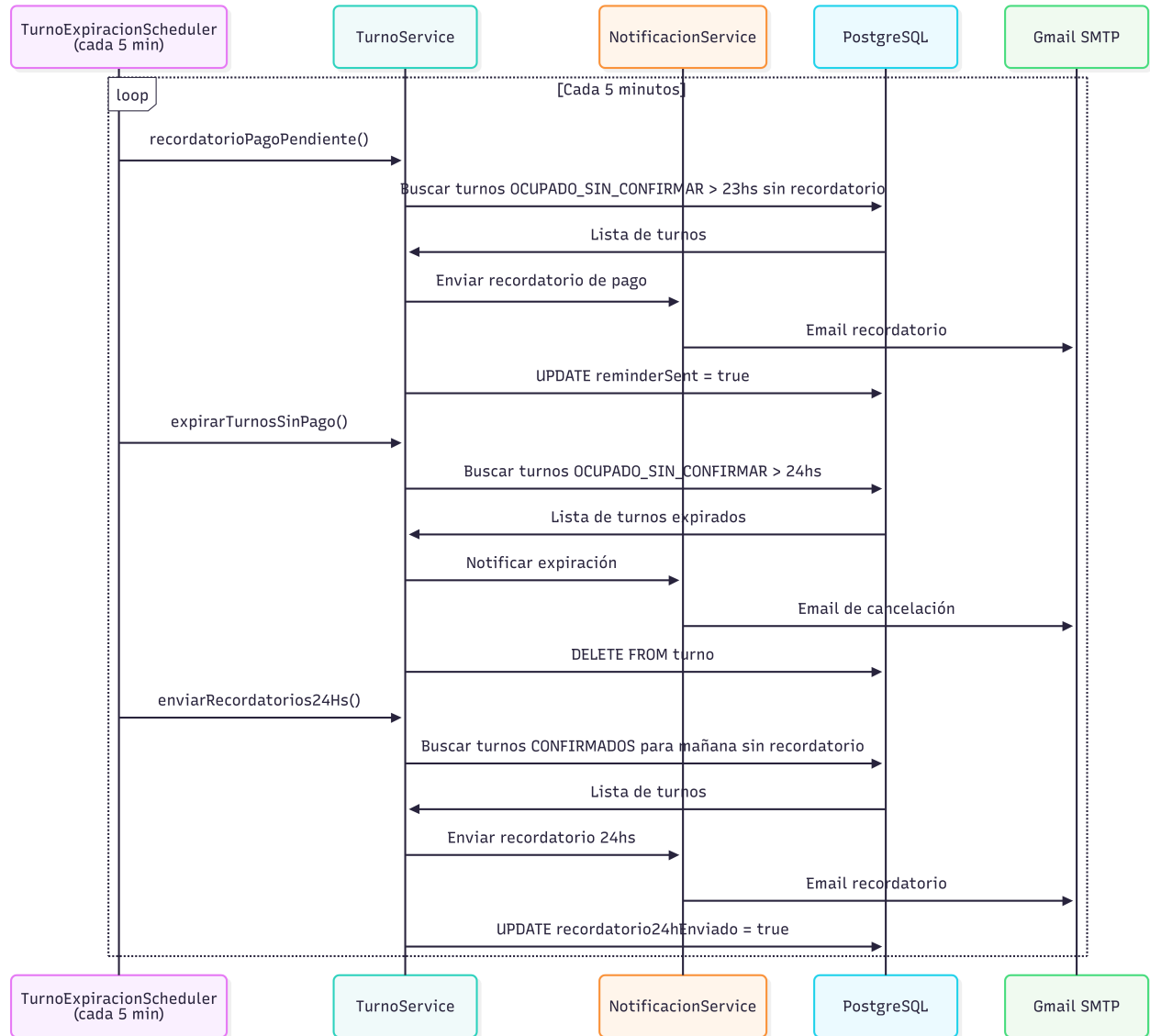
El flujo de autenticación inicia con el login del usuario. El backend valida las credenciales contra la base de datos, emite un par de tokens (access + refresh) y los devuelve al frontend. El access token se incluye en el header `Authorization` de cada solicitud. Cuando expira, el refresh token permite obtener uno nuevo sin intervención del usuario.

Flujo de Creación de Turno



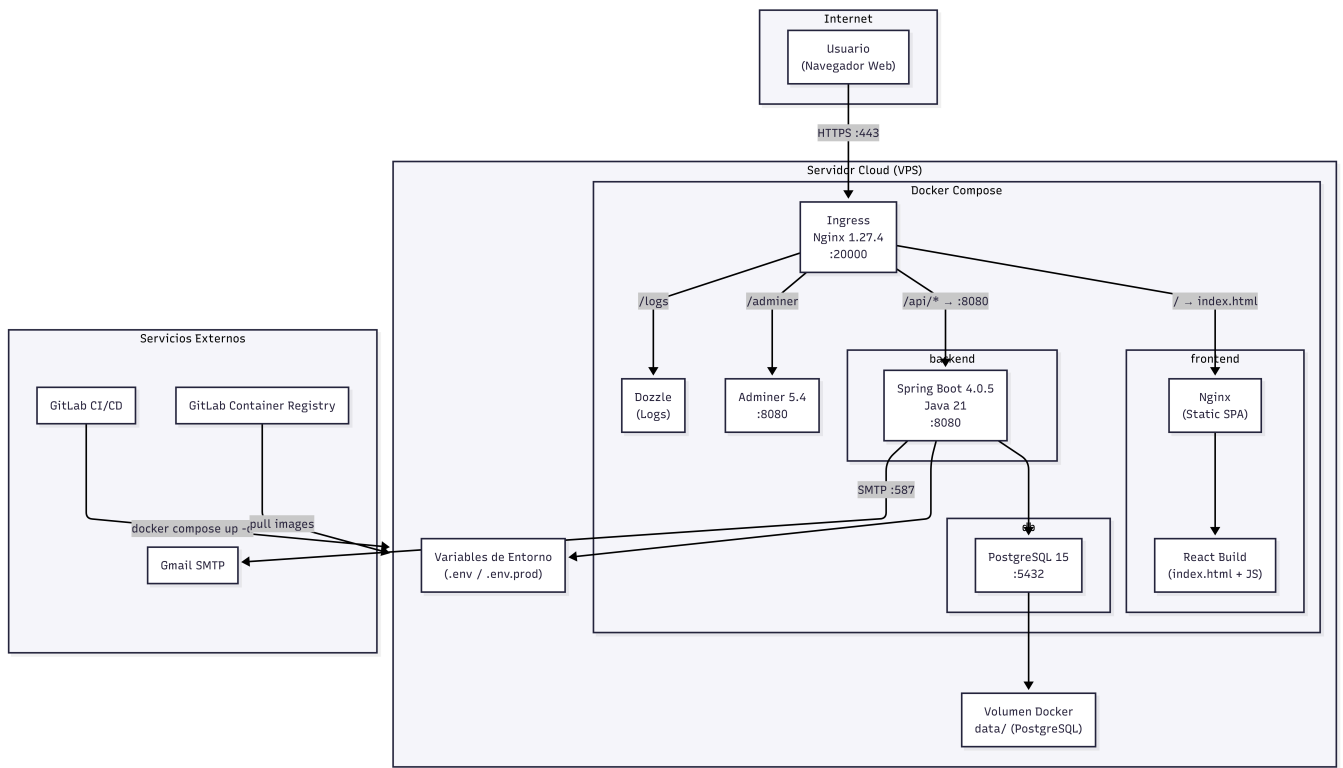
La creación de un turno sigue estos pasos: el cliente selecciona un profesional, un servicio y un horario disponible. El sistema verifica la disponibilidad en la agenda del profesional, registra el turno con estado "pendiente de pago", y opcionalmente envía una notificación por email. El turno queda confirmado solo tras la validación del pago.

Tareas Programadas (Scheduler)



El scheduler ejecuta dos procesos diarios automáticos: (1) cancelar turnos cuyo tiempo de pago haya expirado, notificando al cliente; (2) enviar recordatorios a los clientes con turnos confirmados para el día siguiente. Ambos procesos operan sobre la base de datos de forma transaccional.

5. Vista Física (Despliegue)



El sistema se despliega en un servidor VPS usando Docker Compose. Los servicios se comunican a través de una red interna de Docker:

Servicio	Tecnología	Puerto Interno
Ingress	nginx (reverse proxy)	80 / 443
Backend	Spring Boot 4 (Java 21)	8080
Frontend	nginx + React SPA	80
Base de datos	PostgreSQL 15	5432
Adminer	Adminer 5.4	8080
Dozzle	Dozzle (logs)	8080

El ingress expone un único puerto al exterior y enruta según la ruta: `/api` → backend, `/adminer` → Adminer, `/logs` → Dozzle, el resto → frontend. El frontend se sirve como contenido estático desde un contenedor nginx separado.

Relación entre las Vistas

Las cinco vistas no son independientes, sino que se complementan:

- Los **escenarios** (vista 1) ejercitan los componentes de la **vista lógica** (vista 2) y los **procesos** (vista 4).
- La **vista de desarrollo** (vista 3) muestra cómo se materializan los componentes lógicos en el código fuente.

- La **vista física** (vista 5) indica dónde se ejecutan los procesos y cómo se despliegan los artefactos de desarrollo.

Esta correspondencia permite verificar la consistencia arquitectónica y facilita la comunicación entre los distintos roles del proyecto.